Laboratories Report

Lab 1:

The goal of the first lab was mainly to schedule the tasks of the Robot Agent and evaluate it.

1. The schedule

Periods:

To design our execution schedule and measure the various task execution times, we have considered the following important information from the Compendium:

1. *The Avoid task must be run at least every 100-150ms to avoid overheating the motors.*
   ⇨ The maximum period for the mission task is then 100-150ms.
2. *The time from when a victim is read by the RFID reader to when a message is sent must be less than 1300ms.*
   ⇨ The maximum period for the Refine-Control-Report-Communicate tasks is 1300ms.
3. *If a stop command is received from the mission control, then the robot must stop within 1300ms.*
   ⇨ The maximum period for the mission task is 1300ms.
4. *The robot will only be allowed to communicate every 1000ms. (This is only for Lab 2, but we have chosen to take this into consideration here to simplify the implementation of Lab2).*
   ⇨ The minimum period for the communication task is 1000ms.
5. *If the control task is run more often than every 300ms, it will only accumulate more localisation error.*
   ⇨ The minimum period for the control task is 300ms

After taking into account those information, we have established the minimum and maximum periods for each task in the **Table *1***.

Measures:

For the measurements, we have tested each task with different scenarios as advised in the Compendium, e.g. lifting the robot, making him pass on multiple victims quickly, looking at when it is bumping multiple times in a row, etc. Each time with a different task order.

Each execution time was obtained, by subtracting the precise time just before and just after the task, to see its duration. For each scenario, we have taken multiple measurements and we kept each time, the worst, to simulate the worst case. Our exact worst measurements are reported in the **Table *1***.

WCET:

Our Worst-Case Execution Times were estimated with the rule of thumb of adding 20% to our worst measurements and rounded for convenience. Thus, we took a minimum of 1ms, and all decimal result we rounded to the next integer (i.e. for the *refine task*: $12.072 * 1.20 = 14.4864 \approx 15$). The *WCET* of each task is summed up in the last column of **Table *1***.

Task	Period Min (ms)	Period Max (ms)	Measure (ms)	WCET (ms)
Mission	0	1300(3)	0.174	1
Navigate	0	1300(2)	0.350	1
Refine	0	1300(2)	12.072	15
Report	0	1300(2)	0.001	1
Control	300(5)	1300(2)	19.43	24
Avoid	0	100-150(1)	18.82	23
Communicate	1000(4)	1000(3)	6.864	9

Table 1: Recap of the Period, measurements and WCET for our schedule

Minor and Major Cycle:

When designing our minor and major cycles, we first looked at the important information on the periods choosing and we tried several task orderings to find the one with the best performances. We thus took 100ms for the period of the avoid task and 600ms for the control Task.

Then for the **major cycle** we got: $\mathrm{LCM}(300, 100, 1000) = 3000$. Therefore, our major cycle would be 3000ms ⇔ 3s. And for the **minor cycle** we got: $\mathrm{GCD}(300, 100, 1000) = 100$. So, our minor cycle would be 100ms ⇔ 0.1s which means that we will have 30 minor cycles in each major cycle.

While we make sure to read the information about lab 2 to be able to keep the same schedule for both labs, we also tested the best ordering for the tasks and we looked into the condition of ordering of the tasks to avoid overrides, and we got the following sequence:

Communicate => Refine => Report => Mission => Control => Avoid => Navigate => Idle

Where the *Communicate* task is only used every second, i.e. in minor cycles 1, 11 and 22, see ***Figure 2*** and **Figure 1**.

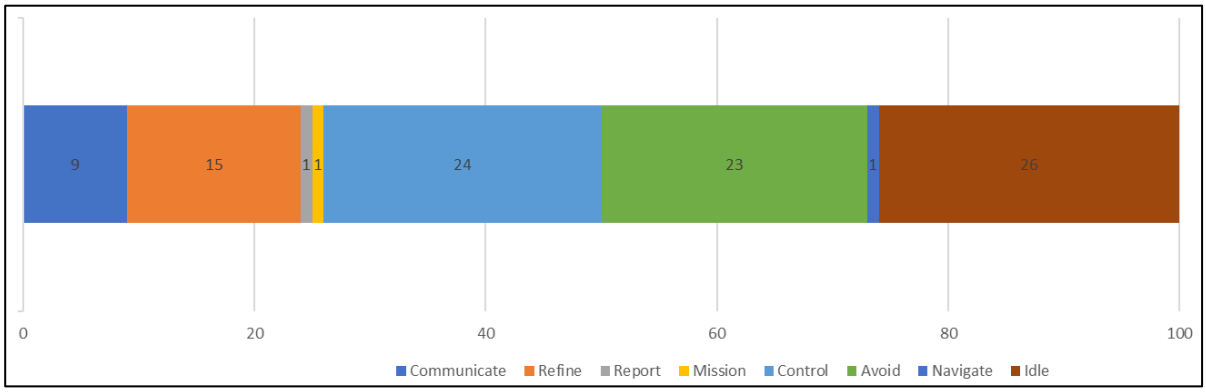

Figure 1: Minor Cycle 1

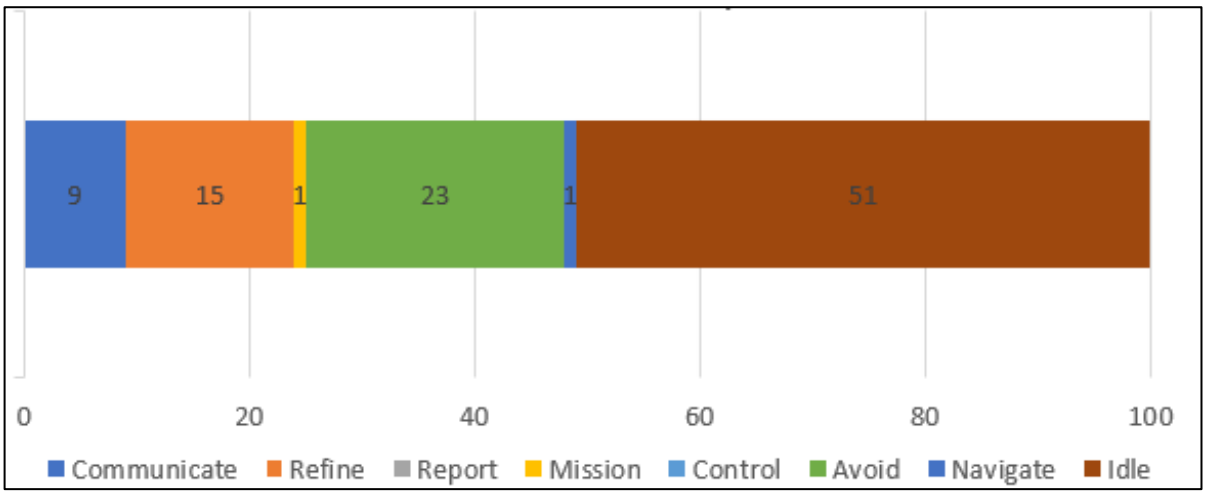

Figure 2: *Minor Cycle 11 and 21*

The *Control* task is only used every 600ms, i.e. in minor cycles 1, 7, 13, 19, 25, see **Error! Reference source not found.**. All other minor cycles are the same without the two tasks cited above, see **Figure 4**.

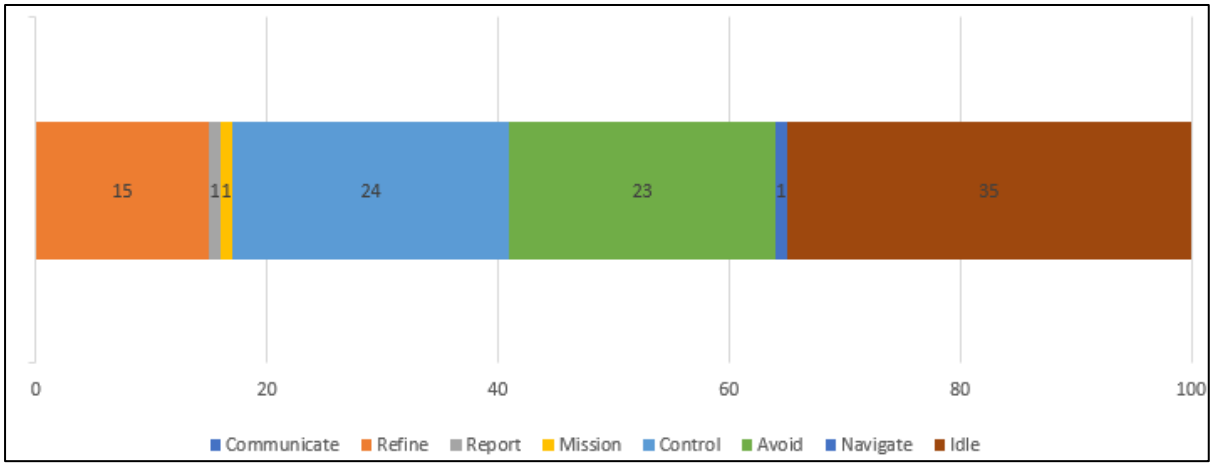

Figure 3: Minor Cycles 7, 13, 19, 25

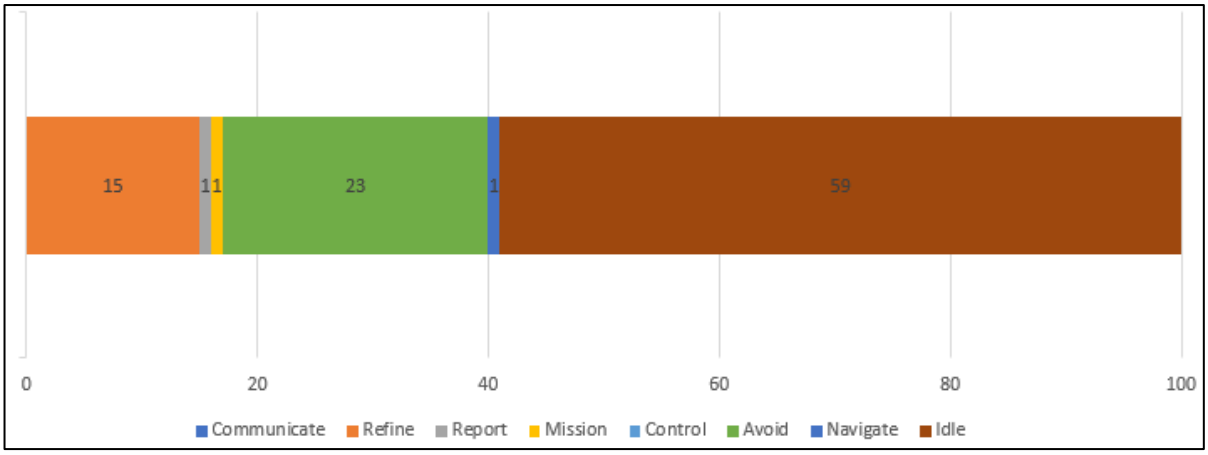

Figure 4: All other minor cycles

2. Schedule Performance

Cycle Overruns:

We measured the time between the first and the last task and we never observed any duration exceeding the minor cycle 100ms. It seems logical since we have at least 26ms of margin for the first minor cycle, see **Figure 1** and most of the time 59ms of margin **Figure 4**. Which coupled with the margin we already took when establishing the WCET make the system very safe from overruns.

Victim detection accuracy:

As for the victim detection accuracy, we computed the Euclidian distance between the coordinates measured by the robot and the ones given in the pdf and the average difference was **235mm** for 10 experiments with 10 victims measured in each.

Lab 2:

The second Lab was about using communication between robots and the mission control in a specific allocated timeslot.

1. The Communication

To make our robot communicate on our dedicated timeslot, we first had to know in which timeslot it was and how to start the communication in it. As there were 8 robots, and each robot needed to communicate every second. It would only have $\frac{1s}{8} = 0.125s$ to communicate. Therefore, we figured that the robot 1 could communicate between 0 and 0.125 of each second, the robot 2 could between 0.125 and 0.250 and so on. Depending on our robot, we could then just pick the corresponding slot as presented in **Table *2***.

	Slot start	Slot end
Robot 1	0.000	0.125
Robot 2	0.125	0.250
Robot 3	0.250	0.375
Robot 4	0.375	0.500
Robot 5	0.500	0.625
Robot 6	0.625	0.775
Robot 7	0.750	0.875
Robot 8	0.875	1.000

Table 2: Each robot and its corresponding slot

Then we just had to figure out a way to start the communication during our slot, and even better to start it at the start of our slot to optimize the quantities of data we could transfer.

We chose the solution of waiting for next full second by looking at the precise time (e.g. if the measure time is 14h52m3.456s, we would wait until 14h52m4.000s). And then we would wait again for the corresponding timeslot of our robot by adding 0.125s as many times as needed (e.g. twice for robot 2). Also, we made sure to put the communicate task and the start of the cycle since Lab1 to be sure that it would be delayed and that we wouldn't lose time in our timeslot.

2. Message Priorities

As said in the compendium, all types of messages are not as critical. That why we privilege the message indicating victim reports and location messages, then the pheromone maps information and lastly the stream data.

It is indicated that the communication between the robot and the mission control is a rate of 153600 bits per second. We had to divide this number by 8 since there are 8 robots and by 8 again to have bytes. Which gives us: $\frac{\frac{153600}{8}}{8} = 2400\ bytes$.

We then, took a safety margin of 10% to avoid as much as possible to exceed our timeslot and cause the stopping of the robot. Which gives us $2400 * 0.90 = 2160\ bytes$ to send.

As, we saw that the critical data were really small compared to the 2160 bytes available. We send them every time as they arrive, whereas we only send pheromone data if there is still some spaces left after the location and victims report messages, and the stream data if there is still spaces after the pheromone data.

3. Communication performances

Data sending:

When applying the division that we just see, we have the following average partition of the sent data:

- Critical data (**Robot localization** + **Victim Reports**): 0.162% of the total data sent
- **Pheromone data**: 92.01% of the total data sent
- **Stream data**: 7.828% of the total data sent

Which seems to be that we want to have according to the compendium. Also, as we have taken the safety margin of 10 %, we have observed that no messages are sent outside of our timeslot.

Number of Stops:

When running the robot alone, it either very seldom or never stops. However, when we use it when other robots are also communicating, we have experienced some stops.

For 6 minutes which is 360s, corresponding to 120 major cycles, we have counted 34 stops which is approximately 0.283 stops per major cycles. Which seems to acceptable for us since the robot was still measuring, reporting and communicating efficiently.

This number of stops might be the result of some errors on the network during the transport of messages and the results of communications overlapping sometimes.